Builder Journal

Developing this Enterprise RAG (Retrieval-Augmented Generation) application was one of the most interesting projects I have worked on. Before starting this project, I had only a basic understanding of how large language models answer questions. While building the application, I learned that a RAG system is much more than simply connecting an AI model. It involves document processing, embeddings, vector databases, retrieval, prompt engineering, and response generation. Although the project was challenging, it helped me improve both my programming skills and my understanding of modern AI applications.

What Worked Well?

One of the best parts of this project was creating a well-organized project structure. I divided the application into different folders for routes, models, services, templates, and static files. This made the project easier to understand and allowed me to work on different features without affecting the rest of the code.

The document upload feature also worked well. I was able to upload PDF files, extract their text, split the content into smaller chunks, generate embeddings using a Sentence Transformer model, and store them in the Chroma vector database. Seeing the chatbot answer questions using the uploaded documents was one of the most satisfying moments of the project because it showed that the entire RAG pipeline was working correctly.

Another thing that worked well was the chat interface. The frontend and backend communicated successfully using JavaScript and Flask APIs. Users could ask questions and receive responses without refreshing the page, making the application feel more interactive and user-friendly.

I also enjoyed working with LangChain because it simplified many tasks. Using prompt templates, retrievers, and Runnable chains helped me connect different components together in a clean and organized way.

What Challenges Did You Face?

Even though many parts worked well, I also faced several challenges during development.

The biggest challenge was understanding the complete RAG workflow. At first, I was confused about how document chunking, embeddings, vector databases, retrievers, and language models worked together. I spent a lot of time reading documentation and watching tutorials before I fully understood the process.

Another challenge was setting up the development environment. Some libraries had compatibility issues, and different versions of LangChain or other packages caused unexpected errors. Sometimes the application stopped working after installing a new package, so I had to carefully manage dependencies and update the code accordingly.

Working with PDF files was another challenge because every document has a different structure. Some PDFs contained extra spaces or formatting problems, which affected text extraction. I also had to experiment with different chunk sizes and chunk overlap values because these settings directly affected the quality of the retrieved information.

I also experienced problems while connecting the frontend with the Flask backend. There were issues with API routes, HTTP methods, JSON responses, and JavaScript fetch requests. Some errors were simple, such as incorrect URLs or missing parameters, but finding them still took time.

How Did You Debug Problems?

Whenever I encountered a problem, I tried to solve it step by step instead of changing everything at once.

The first thing I did was read the error messages shown in the Flask terminal. Most of the time, the traceback clearly showed where the problem was occurring. I also used print statements to display important variables such as uploaded filenames, extracted text, generated embeddings, retrieved documents, and model responses. This helped me understand whether each stage of the pipeline was working correctly.

For frontend issues, I used the browser's Developer Tools. The Console tab helped me identify JavaScript errors, while the Network tab showed whether API requests were reaching the backend successfully. This made it much easier to fix communication problems between the frontend and backend.

When something in the RAG pipeline failed, I tested every component separately. First, I checked whether the PDF text was extracted correctly. Then I verified the chunking process, embedding generation, vector storage, document retrieval, prompt creation, and finally the LLM response. Breaking the project into smaller parts made debugging much easier.

Whenever I couldn't solve a problem on my own, I referred to the official documentation of LangChain, ChromaDB, Flask, and Sentence Transformers. Reading examples and comparing them with my own code often helped me find the solution.

What Would You Improve?

If I continue developing this project, there are several improvements I would like to make.

First, I would add user authentication so that every user can upload and manage their own documents securely. This would make the application more suitable for real-world use.

Second, I would support additional document formats such as Word, PowerPoint, Excel, and text files instead of only PDFs. This would make the system more flexible for different users.

I would also improve the retrieval process by using better embedding models and reranking techniques to increase the accuracy of answers. Adding conversation memory would also improve the chatbot because it would remember previous questions and provide more natural conversations.

Finally, I would deploy the application on a cloud platform so that it could be accessed online. I would also optimize the application to handle larger document collections and multiple users at the same time.

What Did You Learn About RAG?

This project taught me many new concepts that I had never implemented before. I learned that Retrieval-Augmented Generation is a combination of information retrieval and language generation. Instead of relying only on the AI model's built-in knowledge, the system first searches for relevant information from uploaded documents and then uses that information to generate accurate answers.

I also learned the importance of embeddings and vector databases. Before this project, I only knew about traditional databases. Now I understand how embeddings represent the meaning of text as vectors and how similarity search retrieves the most relevant document chunks.

Another important lesson was prompt engineering. I realized that the quality of the prompt has a significant impact on the quality of the final response. Well-structured prompts produce much more accurate and useful answers.

Overall, this project greatly improved my understanding of AI application development. Besides learning the theory behind RAG, I also gained practical experience using Flask, LangChain, ChromaDB, Sentence Transformers, JavaScript, and REST APIs. Completing this project increased my confidence in building real-world AI applications and gave me a much better understanding of how enterprise RAG systems work.